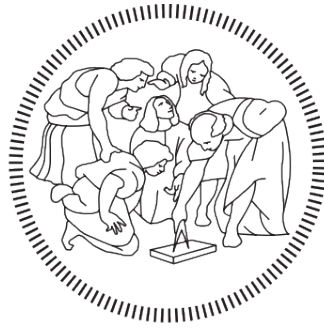




POLITECNICO

MILANO 1863

054323 – Software Engineering 2 2025
DD

Design Document

Best Bike Paths

Juan Sebastian Sarmiento Pulido- 11153648
Pedro Augusto Villarinho dos Santos - 11108811
Esteban Lopez Barreto - 11154567

Wednesday 7th January, 2026

Contents

1	INTRODUCTION	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions, Acronyms, Abbreviations	2
1.4	Revision History	2
1.5	Reference Documents	2
1.6	Document Structure	2
2	ARCHITECTURAL DESIGN	3
2.1	Overview: High-level components and their interaction	3
2.2	Component view	6
2.3	Deployment view	10
2.4	Runtime view:	10
2.5	Component interfaces	10
2.6	Selected architectural styles and patterns	14
2.7	Other design decisions	14
3	USER INTERFACE DESIGN	14
4	REQUIREMENTS TRACEABILITY	22
5	IMPLEMENTATION , INTEGRATION & TESTING	22
5.1	Overview	22
5.2	Implementation Plan	23
5.3	Component Integration and Testing	23
5.4	System Testing	23
5.5	Additional Specifications on Testing	23
6	EFFORT SPENT	23
7	REFERENCES	23

1 INTRODUCTION

1.1 Purpose

1.2 Scope

1.3 Definitions, Acronyms, Abbreviations

1.4 Revision History

- Version 0.1: Setup
 - Created first layout.
- Version 1.0: First release
 - Section 1 added.
 - Section 2 added.
 - Section 3 added.

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the webeep page of the Software Engineering II course.

1.6 Document Structure

1. Introduction: it aims to give a brief description of the project. In particular it's focused on the reasons and the goals that are going to be achieved with its development, as previously explained in the RASD.
2. Architectural Design: : This part of the document describes at different levels of granularity how the architecture is designed and how it is intended to be deployed and working in a production environment.
3. User Interface Design: This section shows the wire frame that constitutes the platform UI
4. Requirements Traceability: in this section it is explained how the goals defined in the RASD document are satisfied by the interaction between the requirements and the component described in the architectural design section.

5. Implementation, Integration and Test Plan: this section provides a description of the plan to implement subsystems and components, likewise, a description of the plan for the integration of the components and testing of the integration.
6. Effort Spent: shows the effort spent to write this document by each participant
7. Bibliography: it contains the references to any documents and software used to write this artifact and to develop the diagrams and different figures exposed in it.

2 ARCHITECTURAL DESIGN

2.1 Overview: High-level components and their interaction

The BBP platform uses a hybrid architecture that combines microservices with message brokers to create a system that can handle both regular user requests and real-time sensor data streaming, with WebSockets being used as the technology that makes the real-time communication possible

Presentation Layer

The presentation tier includes a Desktop Web App and a Mobile Web App that users interact with. The Desktop Web App handles normal operations like searching for paths and viewing routes using standard HTTP requests. The Mobile Web App does the same thing but also establishes WebSocket connections to continuously send sensor data to the server in real-time, which powers the automatic path recording feature.

Application Layer

The application tier sits between the presentation layer and the data layer. It includes the API Gateway. When a request comes in from either web application, the API Gateway receives it, checks if the user is authorized to perform that action, and then directs it to the appropriate backend service. The API Gateway also maintains the WebSocket connections with the Mobile Web App, allowing it to continuously receive sensor data streams. The Load Balancer works with multiple copies of the API Gateway to distribute incoming requests across them, ensuring that no single gateway instance gets overwhelmed.

The backend is composed of several independent microservices, each following the single responsibility principle by handling a specific functionality.

The Automatic Path Recording Service is a key component that directly receives the continuous stream of sensor data coming from the Mobile Web App through the WebSocket connection. Once it receives this sensor data, the Automatic Path Recording Service publishes it to a message broker, which then sends this data to other services that need it. The Obstacle Detection Service subscribes to this sensor data stream and uses it to detect obstacles along the path. The Statistics Service also subscribes to the sensor data to analyze metrics like velocity, distance and other data. Additionally, the Path Construction Service uses the sensor data segments to construct actual bike paths by piecing together

the recorded route segments into complete paths that can be stored and reused as Bike Paths.

The Authentication Service manages user login and authorization, the Search Service helps users find paths and routes efficiently while also scoring them based on the users' submissions, and the Path Service handles both the creation of manually entered paths and the reception of automatically recorded paths from the Automatic Path Recording Service. These microservices communicate with each other using REST APIs for direct requests between services.

Data Layer

Data is stored in three separate databases. PathDB stores both automatically recorded paths and manually created paths, maintaining all path information and route data. SearchDB stores indexed path data and scoring information used for efficient path searching and retrieval. UsersDB stores user credentials and profile information. All the databases are replicated across multiple servers so that if one server fails, the system continues working.

External Services

The system also integrates with external services. The Maps Provider supplies geographic and mapping data needed for route computation. The Weather Service Provider delivers real-time weather information that can influence path recommendations. The Mobile Sensor Manager and Web GPS Receiver collect location and sensor telemetry from user mobile devices, with this data flowing to the backend through the WebSocket connection established by the Mobile Web App.

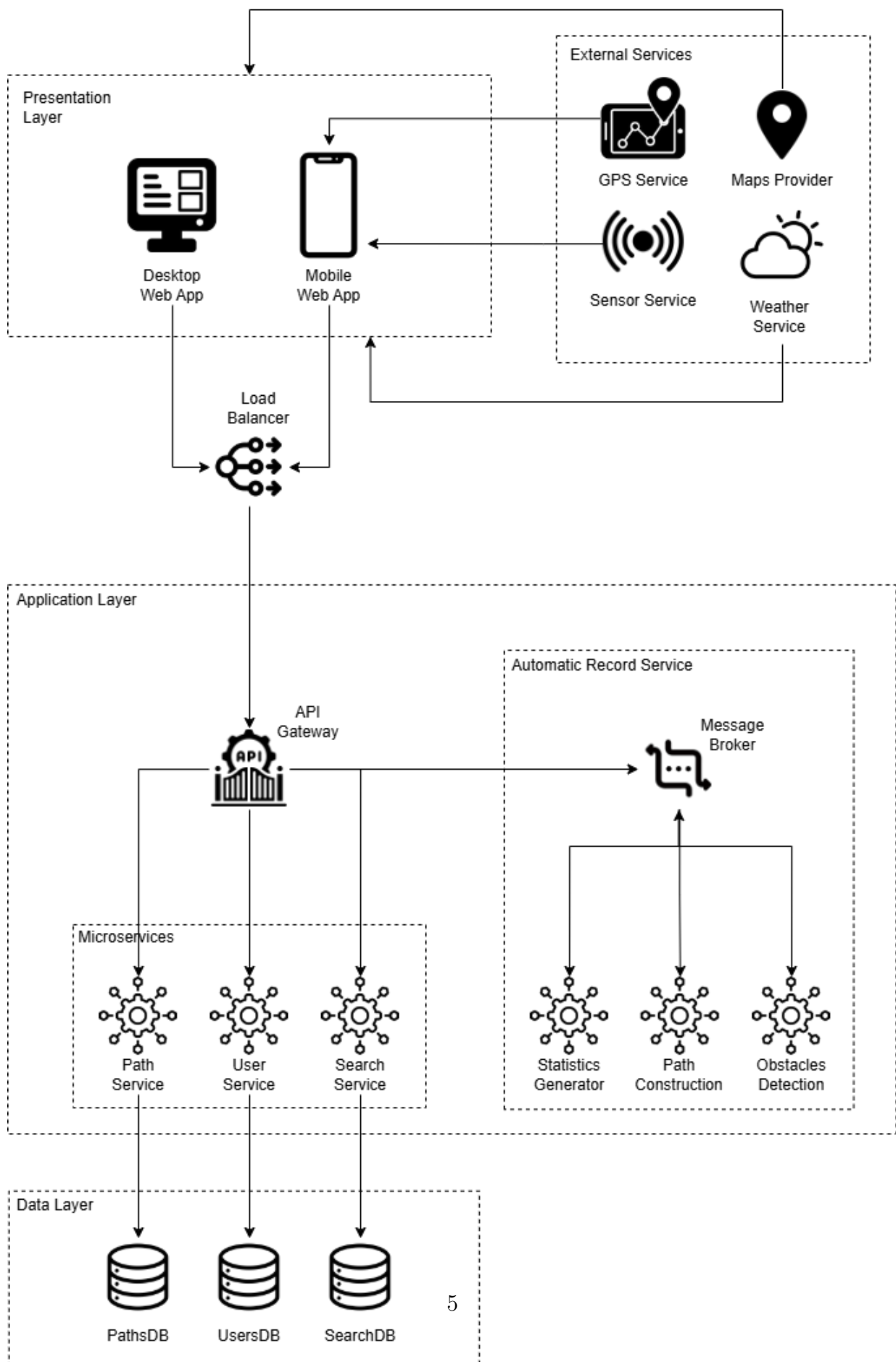


Figure 1: Informal View.

2.2 Component view

The BBP system is composed of various components, each responsible for specific functionalities. These components interact with one another to deliver path recording, storage, analysis, and discovery capabilities. Additionally, the system integrates external services to support these core functionalities.

Presentation Layer

- **Desktop Web App**

This is the component through which desktop users directly interact with the BBP system. It provides a user interface for searching paths, viewing route information, and performing standard operations. The application sends HTTP requests to the API Gateway, which routes them to appropriate backend services. The Desktop Web App is optimized for larger screens and can handle the manual recording of Bike Paths.

- **Mobile Web App**

This is the component through which mobile users interact with the BBP system. It provides the user interface for searching paths and viewing routes, same as the Desktop Web App. The Mobile Web App enables the Automatic Path recording feature establishing WebSocket connections with the API Gateway to stream real-time sensor data from the user's mobile device. The Mobile Web App is also optimized for touch interactions and mobile screens.

Infrastructure Components

- **API Gateway**

This component serves as the central entry point for all client requests from the presentation layer. It receives HTTP requests from both the Desktop Web App and Mobile Web App, and manages WebSocket connections for real-time sensor data streaming from the Mobile Web App. The API Gateway handles request routing to the backend services performing authentication through the Authentication Service. Multiple copies of the API Gateway are deployed to handle incoming traffic at scale.

- **Load Balancer**

This component distributes incoming requests across multiple copies of the API Gateway to ensure no single gateway instance becomes a bottleneck and get overwhelmed. It monitors the health of gateway instances and directs traffic only to available instances. The Load Balancer enables horizontal scaling by allowing additional API Gateway copies to be added as demand increases.

- **Message Broker**

This component enables asynchronous communication between the Automatic Path recording service and its sub components. The Automatic Path Recording Service publishes processed sensor data to the message broker, and other services such as the Path Construction Service, Obstacle Detection Service, and Statistics Service subscribe to this data stream. This way, services are decoupled from each other and can process and send their corresponding data without direct synchronous dependencies.

Application Layer

• Automatic Recording Service

This component receives and processes the continuous stream of sensor data coming from the Mobile Web App through the WebSocket connection. It collects GPS coordinates and sensor metrics from user devices and validates the incoming data. The service publishes raw sensor data to the message broker, where other services consume it to perform their specialized analysis. The service manages the complete recording lifecycle by collecting processed data from the Path Construction Service, Obstacle Detection Service, and Statistics Service through the message broker. It aggregates the processed data from these services to construct a complete bike path that includes path coordinates, detected obstacles, and statistical metadata. Once the recording session is complete, the Automatic Path Recording Service sends the constructed bike path to the Path Service, which stores it in the database.

• Path Construction Service

This component subscribes to the sensor data stream published by the Automatic Path Recording Service through the message broker. It constructs actual bike paths by analyzing and processing the GPS sensor data segments. The service pieces together recorded route segments into complete paths that represent actual bike paths traveled by users. The constructed path is published back to the message broker for the Automatic Path Recording Service to aggregate. This service coordinates with the Maps Provider to ensure that constructed paths are geographically valid and properly mapped.

• Obstacle Detection Service

This component subscribes to the sensor data stream published on the message broker by the Automatic Path Recording Service. It continuously analyzes incoming sensor data to identify potential obstacles or hazards by detecting movement anomalies. The service identifies obstacles and includes them in the processed data. The detected obstacles are published back to the message broker for the Automatic Path Recording Service to include in the final bike path. The obstacles are shown at the end of the recording session.

• Statistics Service

This component subscribes to the sensor data stream published on the message broker to analyze usage metrics and behavioral patterns. It processes the incoming sensor data to calculate statistics such as max velocity, distance traveled, elevation changes and total time. The calculated statistics are published back to the message broker for the Automatic Path Recording Service to include in the final bike path.

- **Path Service**

This component manages the storage and retrieval of all path information within the system. It handles two types of paths: automatically recorded paths that are constructed by the Automatic Path Recording Service, and manually created paths that users enter themselves. The service provides path management and also constructs composite paths that are not listed in the search by using path segments from the database and assigning them to corresponding origins and destinations requested. It exposes APIs that allow other services and user-facing applications to access and manage path data. The Path Service maintains its data in the PathDB database, where all path records, path metadata, and path history are stored.

- **Search Service**

This component helps users discover and find paths and routes efficiently within the system. It maintains an index of all available paths and implements searching functionality. The service calculates rating scores for bike paths based on user submissions and prioritizes those who are frequently updated and valued. The Search Service uses the SearchDB database to store indexed path data, search indices, and scoring information.

- **Authentication Service**

This component manages all user authentication and authorization processes within the BBP platform. It verifies user credentials and manages user sessions. The service stores and retrieves user credentials, authentication tokens, and user profile information from the UsersDB database.

Data Layer

- **PathDB**

This database stores all path-related information within the BBP system. It maintains records of both automatically recorded paths and manually created paths, including complete path coordinates, route segments, obstacles and path metadata.

- **SearchDB**

This database stores indexed paths and their corresponding rating scores submitted by users.

- **UsersDB**

This database stores all user-related information including user credentials and authentication tokens.

External Services

- **Maps Provider**

This service supplies geographic and mapping data to the BBP system. The Maps Provider provides mapping visualization for computed routes and recorded paths in the user-facing applications and returns geographic coordinates and map information needed for path computation and display.

- **Weather Service Provider**

This service delivers real-time weather information to the BBP system. The Statistics Service uses weather data to enrich the statistical information collected during automatic path recording, providing context about weather conditions during path recording sessions.

- **Mobile Sensor Manager**

This external component represents the native sensor capabilities available on user mobile devices, including GPS, accelerometers, and gyroscopes. The Mobile Web App accesses these device sensors to gather location and motion data and transmit it through the WebSocket connection to the Automatic Path Recording Service.

- **Web GPS Receiver**

This external component represents the GPS service available on user mobile devices. The Mobile Web App accesses the device's GPS receiver to obtain accurate positioning information. It works with the Mobile Sensor Manager to ensure reliable GPS coordinates for path recording.

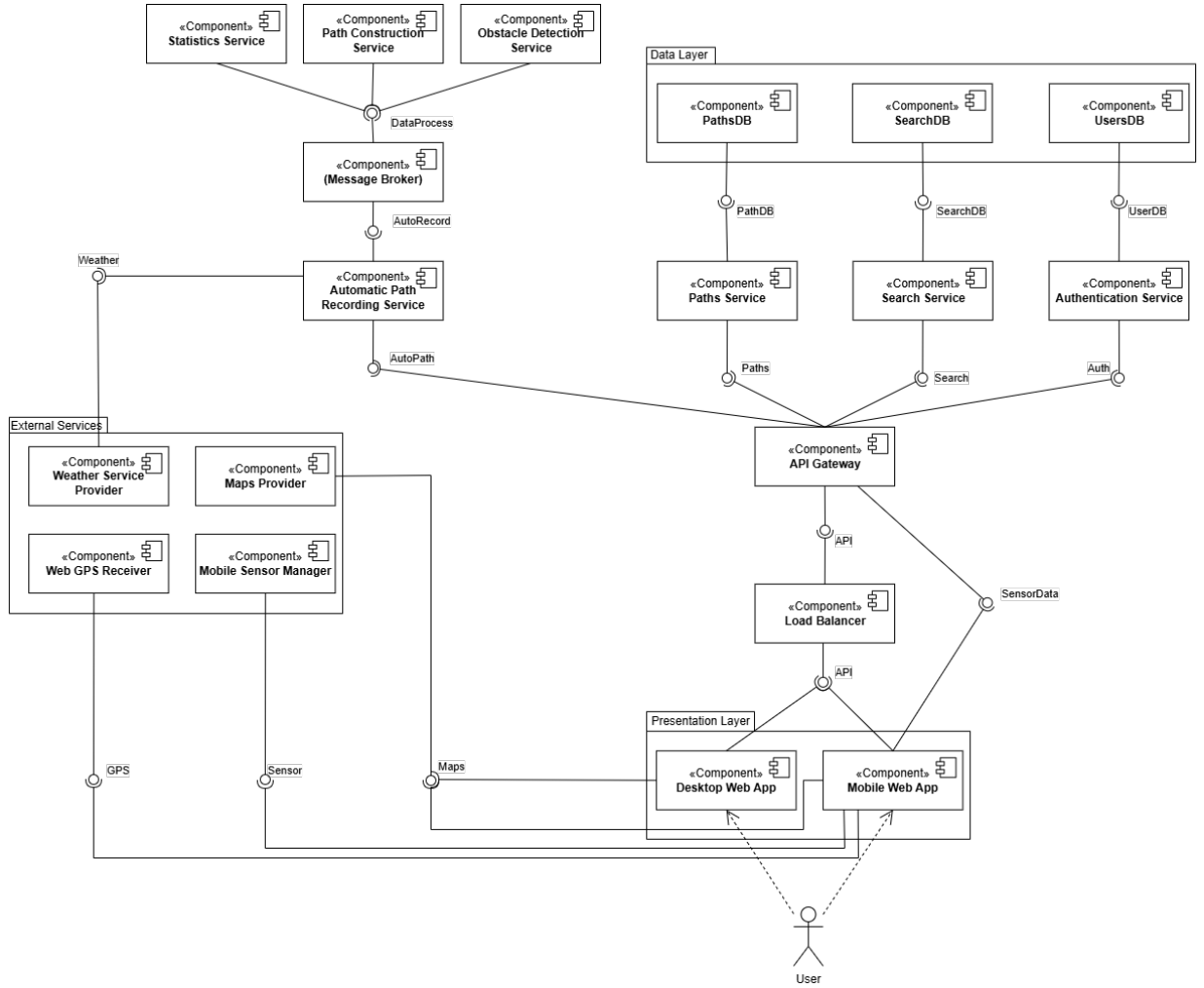


Figure 2: C&C Diagram.

2.3 Deployment view

2.4 Runtime view:

2.5 Component interfaces

This section provides a comprehensive summary of all methods within the interfaces of the Best Bike Paths (BBP) system. The system employs a **hybrid communication architecture**:

- **Internal REST APIs:** For request-response operations between services (e.g., Path Service, Search Service, Authentication Service)
- **WebSocket Protocol:** For real-time bidirectional communication (e.g., sensor data streaming during path recording)
- **Event-Driven Messaging:** For asynchronous processing and inter-service communication

API

- `get_profile(user_id)`
- `get_path(path_id)`
- `get_ownPaths()`
- `get_statistics(path_id)`
- `get_reviews(path_id)`
- `get_searchPaths(origin, destination)`
- `post_register(UserCredentials)`
- `post_login(UserCredentials)`
- `post_createPath(Path)`
- `post_submitReview(path_id, Review)`
- `patch_updatePath(path_id, PathMetadata)`
- `patch_publishPath(path_id)`
- `ws_connect()`
- `ws_disconnect()`

Auth

- `post_register(UserCredentials)`
- `post_login(UserCredentials)`

Paths

- `post_createPath(Path, user_id)`
- `get_path(path_id)`
- `get_ownPaths(user_id, page, limit)`
- `patch_updatePath(path_id, PathMetadata)`
- `patch_updateScore(path_id, new_score)`
- `patch_publishPath(path_id)`
- `post_submitReview(path_id)`
- `get_reviews(path_id)`

Search

- `get_searchPaths(origin, destination)`

SensorData

- `ws_publishRawSensorData(SensorDataSegment, session_id)`

AutoPath

- `ws_startRecording(session_id)`
- `ws_stopRecording(session_id)`
- `ws_publishRawSensorData(SensorDataSegment, session_id)`
- `ws_getPath(session_id)`

AutoRecord

- `ws_heartbeat(session_id)`
- `ws_pathRecordingStarted(session_id, start_time, location)`
- `ws_publishRawSensorData(SensorDataSegment, session_id)`
- `ws_pathRecordingFinished(session_id, end_time)`
- `ws_getStatisticsData(session_id)`
- `ws_getObstaclesData(session_id)`
- `ws_getPathData(session_id)`
- `ws_publishPath(session_id, pathMetadata, statisticsMetadata, obstaclesMetadata)`

DataProcess

- `ws_getRawSensorData(SensorDataSegment, session_id)`
- `ws_publishStatisticsData(session_id, statisticsMetadata)`
- `ws_publishObstaclesData(session_id, obstaclesMetadata)`
- `ws_publishPathData(session_id, pathMetadata)`

Weather

- `get_currentWeather(latitude, longitude)`

Maps

- `get_coordinates(address)`

- `get_address(latitude, longitude)`

PathDB

- `store_path(Path)`
- `query_path(path_id)`
- `query_paths(user_id)`
- `update_path(path_id, PathMetadata)`
- `delete_path(path_id)`
- `store_segment(Segment, path_id)`
- `store_obstacle(Obstacle, path_id)`
- `query_obstacles(path_id)`
- `update_pathScore(path_id, score)`
- `query_reviews(path_id)`
- `store_review(Review)`

PathDB

- `store_user(User, hashed_password)`
- `query_user(email)`
- `query_userById(user_id)`
- `store_session(user_id, token, expires_at)`
- `invalidate_session(token)`

SearchDB

- `query_search(SearchFilter)`
- `update_index(path_id, metadata, score)`

2.6 Selected architectural styles and patterns

2.7 Other design decisions

3 USER INTERFACE DESIGN

Here is shown a mockup of the web application UI. The images depict the main screens of the system, which include a login and registration interface, a home page where users can search bike paths by origin and destination even without an account, once logged it has dedicated sections for recording trips, either manually or automatically, with options available to edit both manual and automatic records like adding streets and/or obstacles. The application also provides a page for visualizing all recorded paths and another for reviewing individual trips shared by other people.

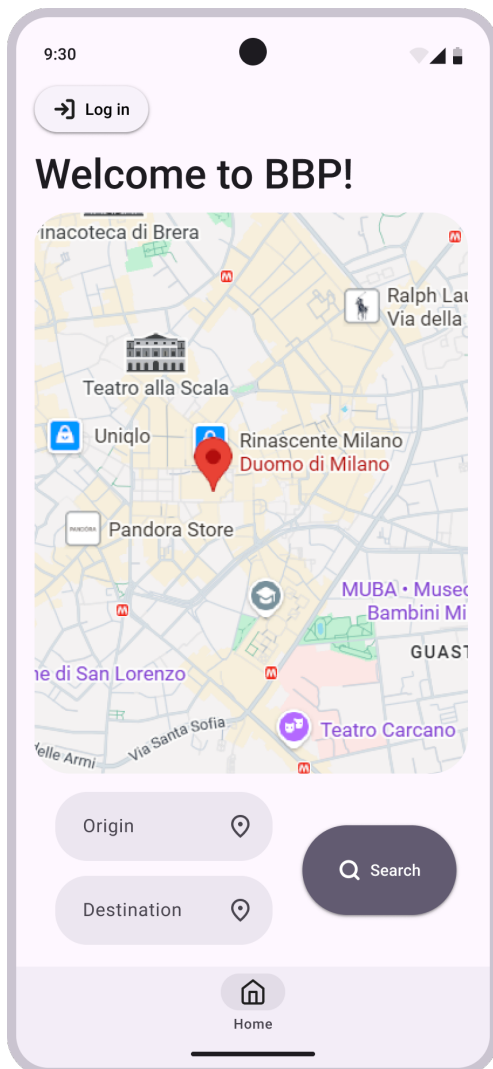


Figure 3: HomePage Not LoggedIn.

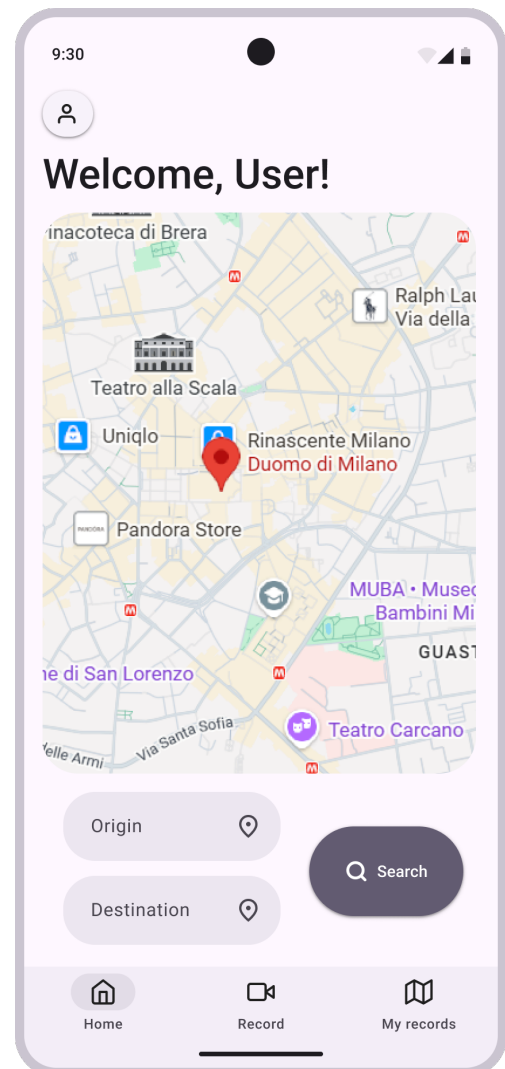
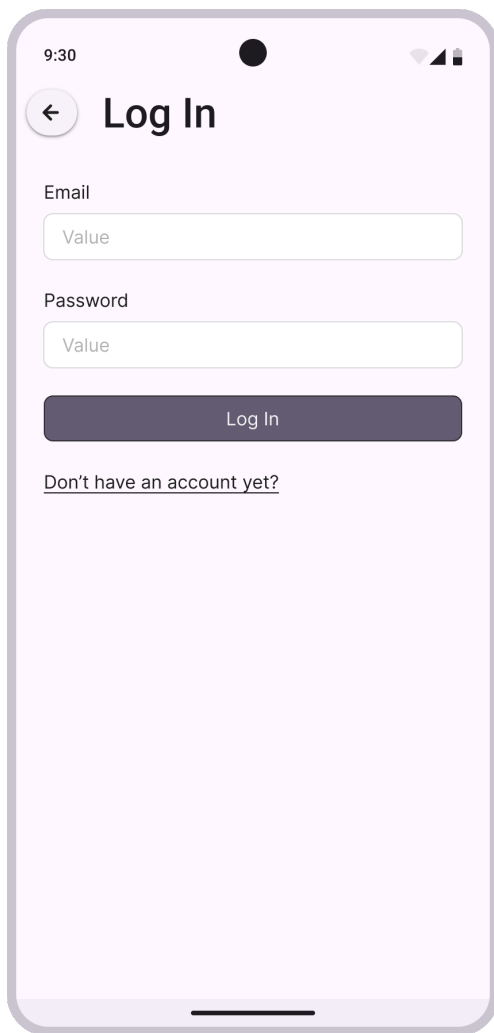
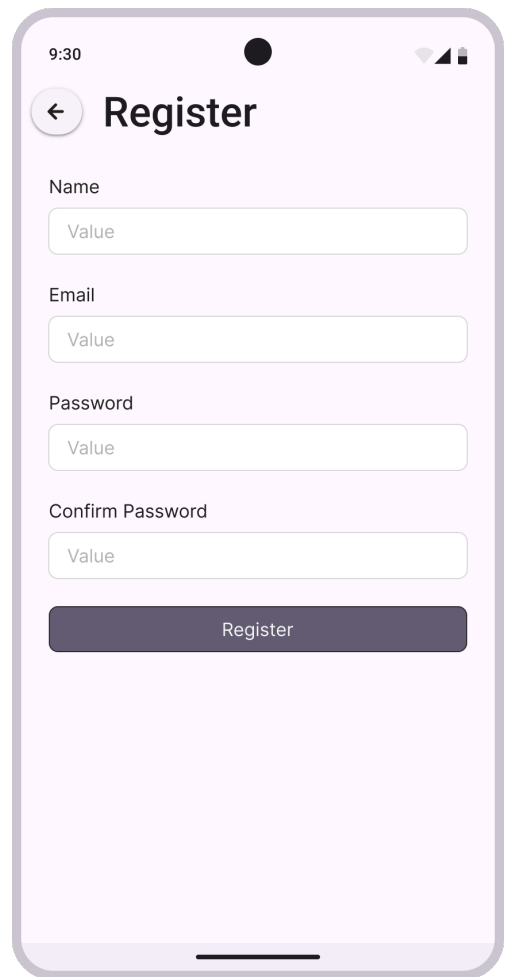


Figure 4: HomePage LoggedIn.



A mobile app login screen with a light purple background. At the top, the status bar shows 9:30, a black circle, and signal icons. Below the status bar is a header with a back arrow in a circle and the text "Log In". The main content area contains two text input fields: "Email" and "Password", each with a light green placeholder text "Value". Below these fields is a dark purple button with the text "Log In". At the bottom, there is a link that says "Don't have an account yet?". The bottom of the screen features a white home indicator bar.

Figure 5: Login Page.



A mobile app register screen with a light purple background. At the top, the status bar shows 9:30, a black circle, and signal icons. Below the status bar is a header with a back arrow in a circle and the text "Register". The main content area contains four text input fields: "Name", "Email", "Password", and "Confirm Password", each with a light green placeholder text "Value". Below these fields is a dark purple button with the text "Register". The bottom of the screen features a white home indicator bar.

Figure 6: Register Page.

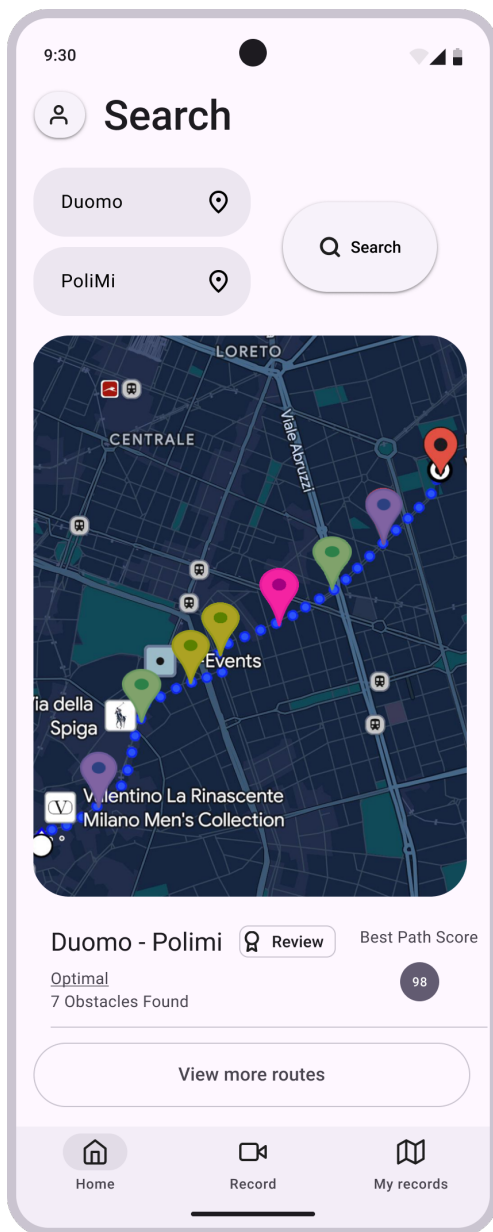


Figure 7: Search page.

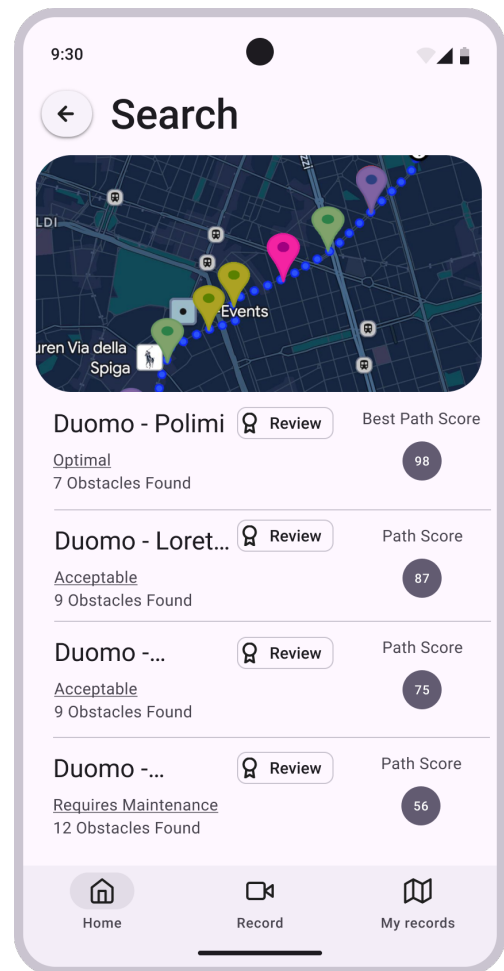


Figure 8: More routes page.

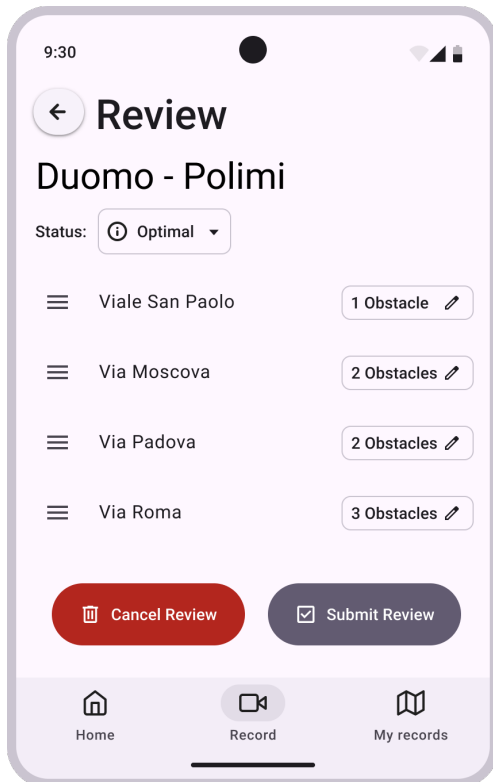


Figure 9: Review page.

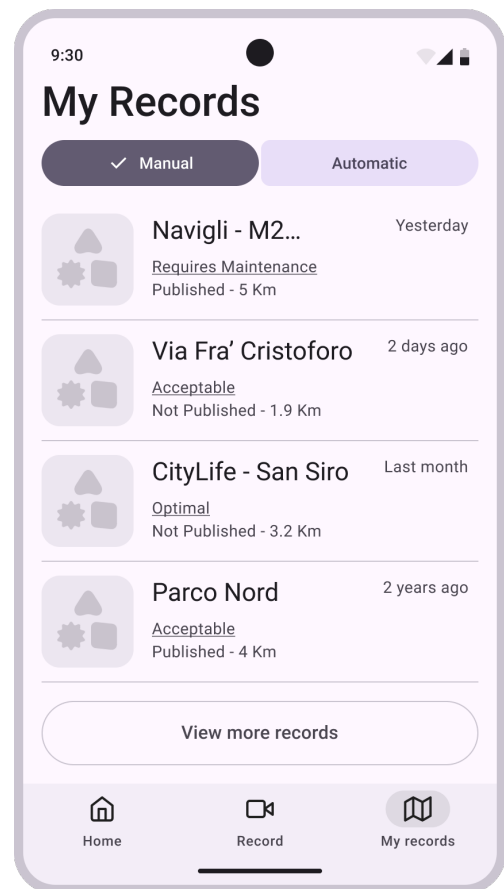


Figure 10: My Manual Records page.

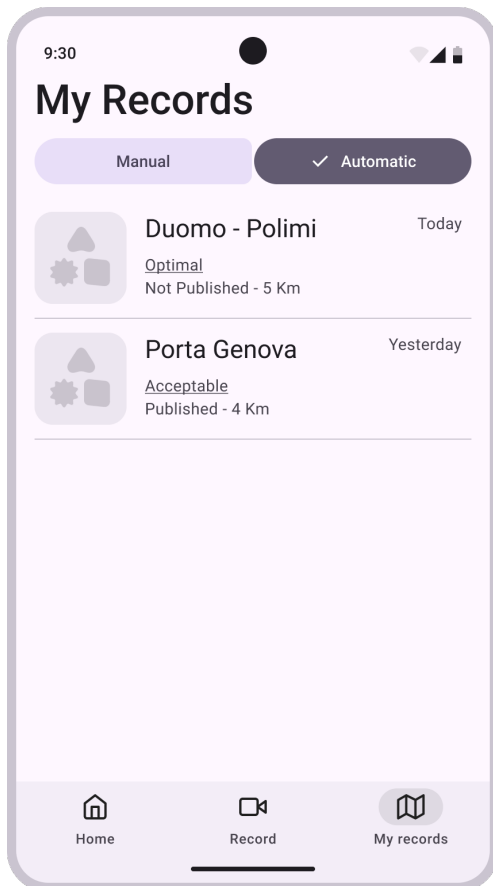


Figure 11: My Automatic Records page.

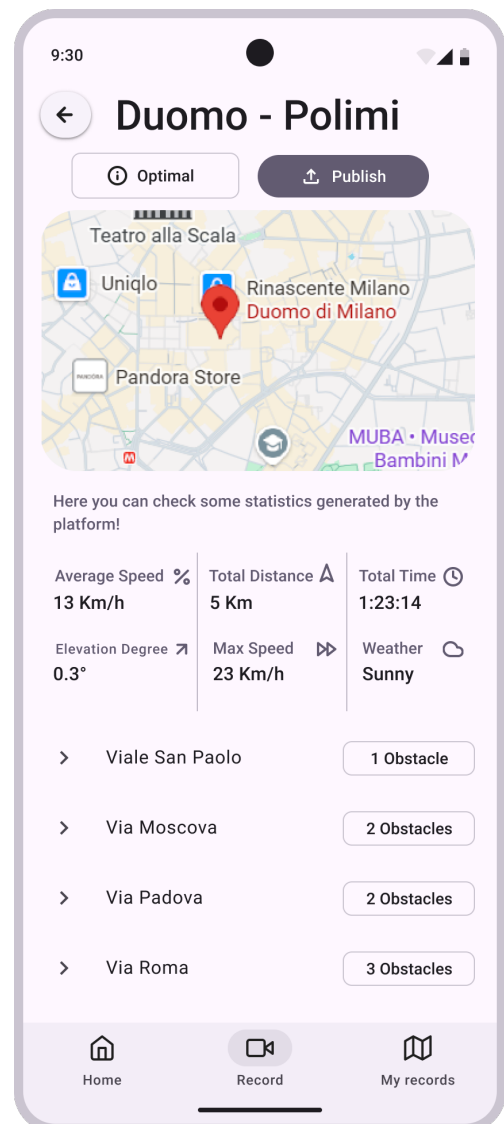


Figure 12: My record page.

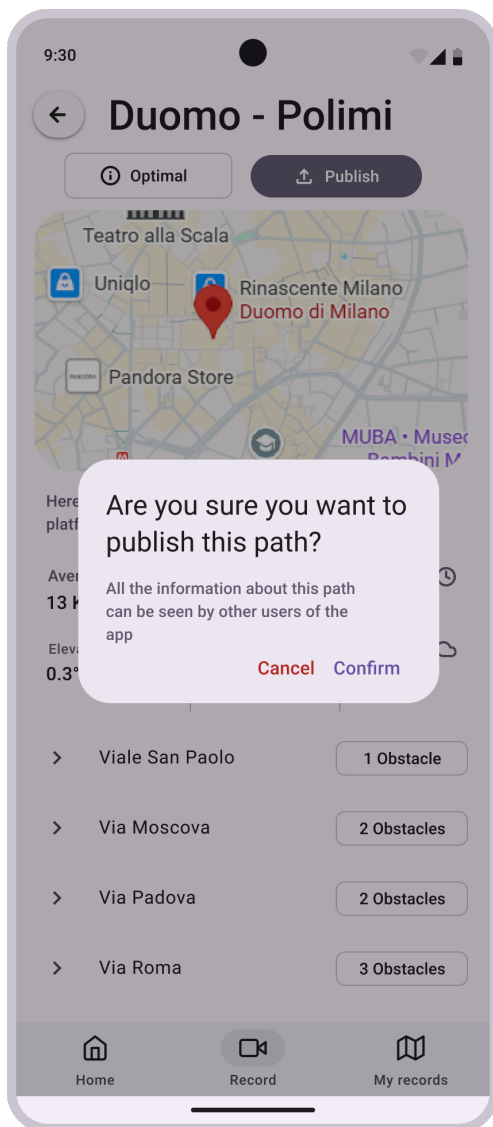


Figure 13: Publish window.

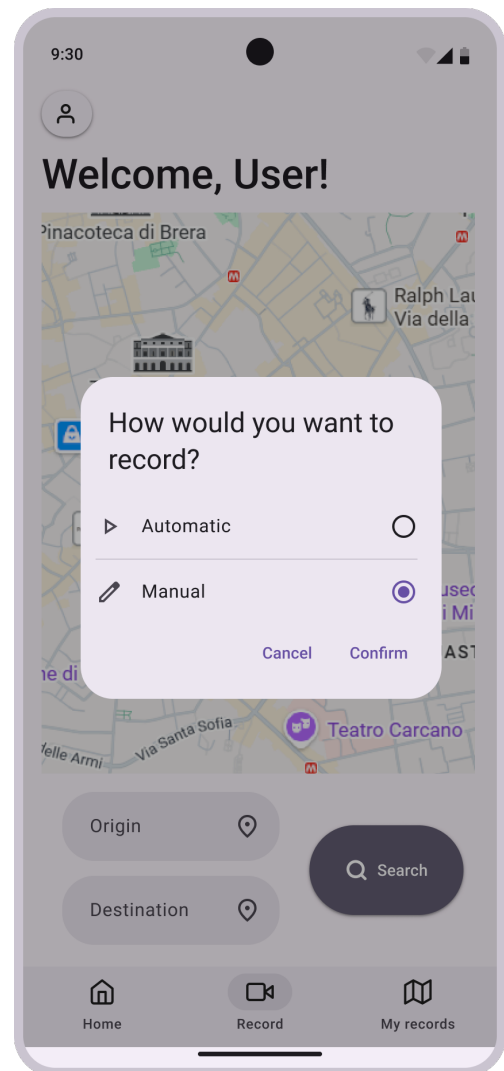


Figure 14: Record window.

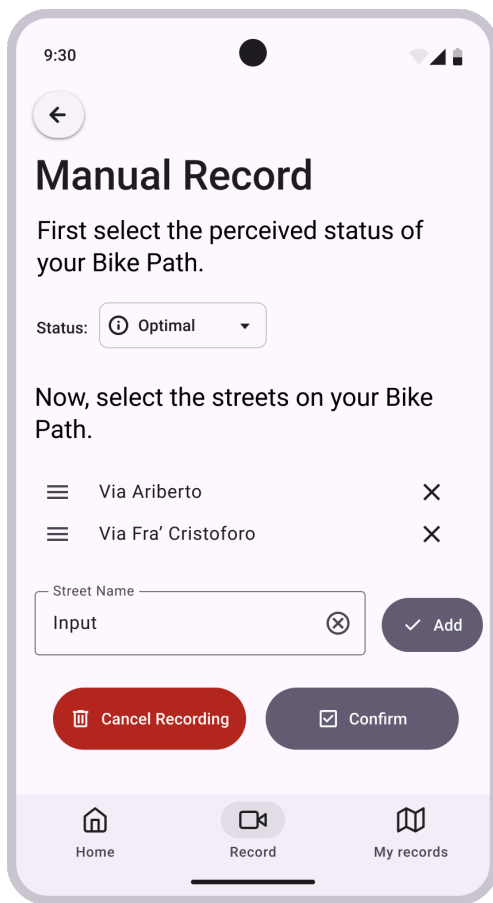


Figure 15: Manual record page.

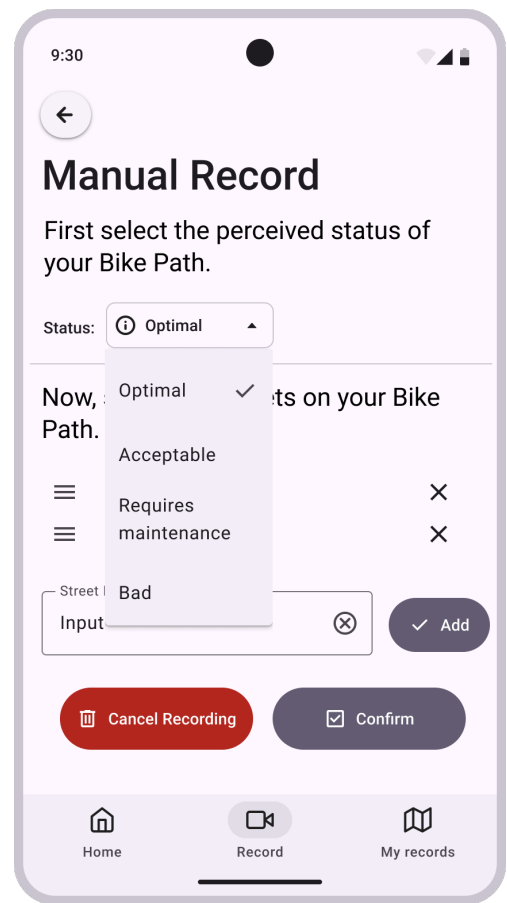


Figure 16: Status dropdown menu.

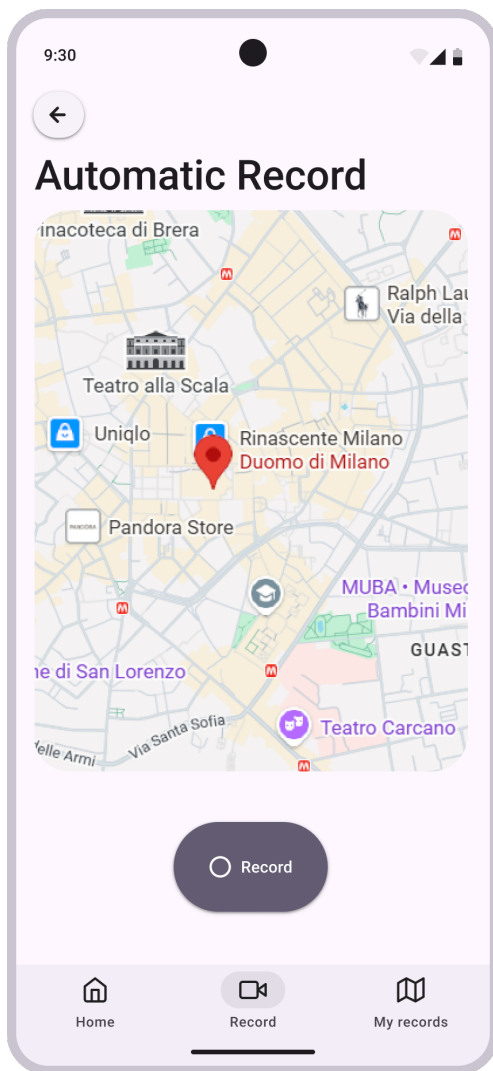


Figure 17: Automatic Record page.

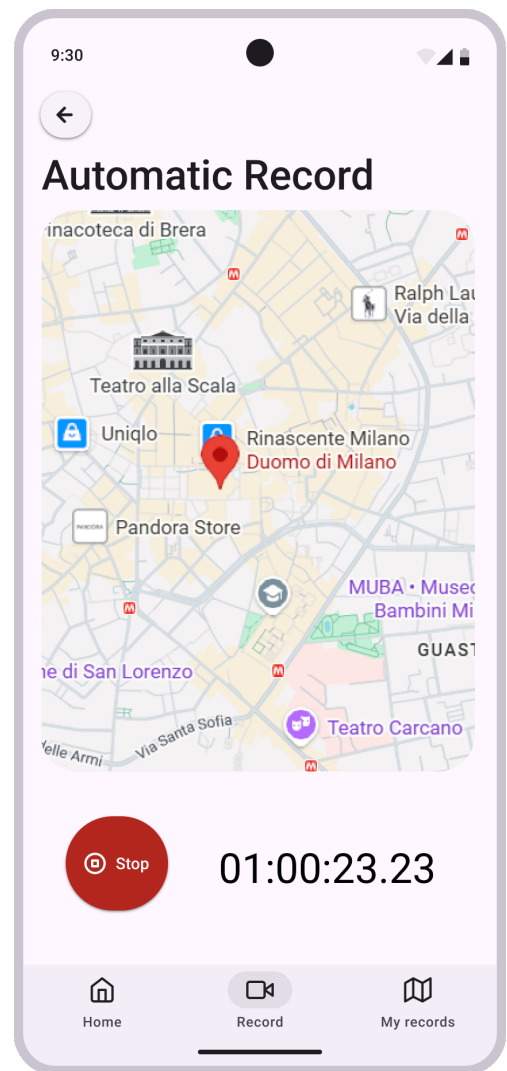


Figure 18: Recording page.

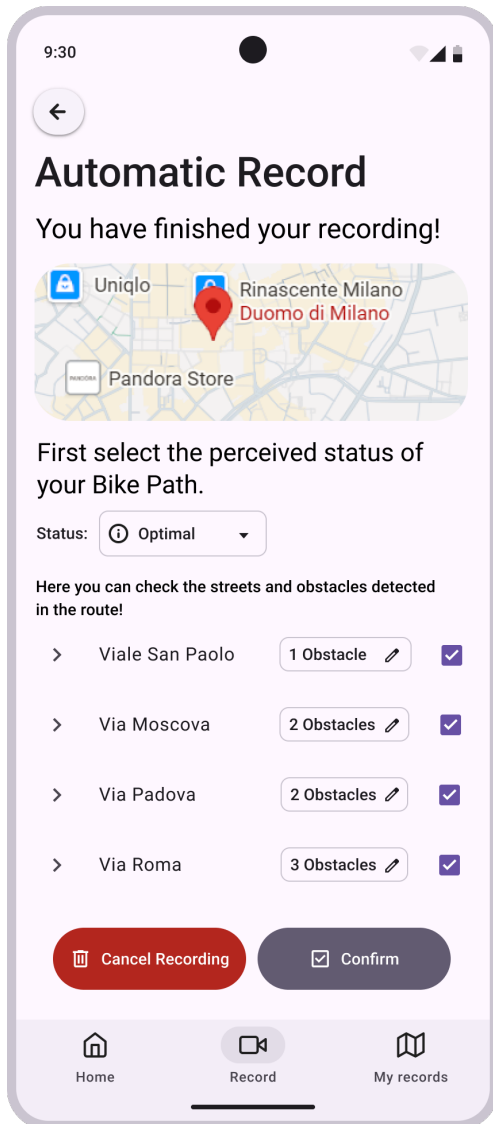


Figure 19: Recording Finish page.

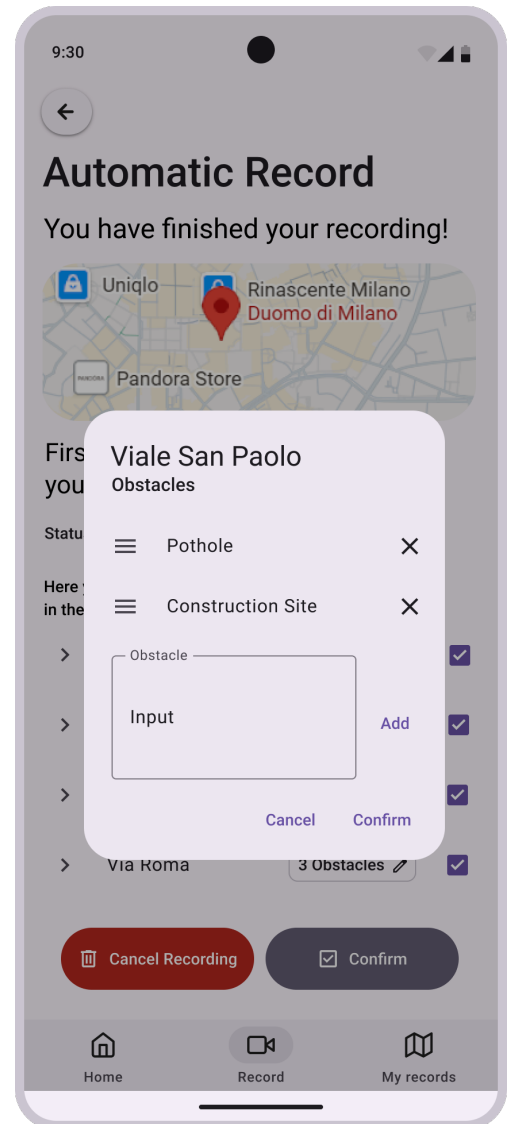


Figure 20: Obstacle Window menu.

4 REQUIREMENTS TRACEABILITY

5 IMPLEMENTATION , INTEGRATION & TESTING

5.1 Overview

This section is dedicated to explaining the implementation, integration and testing strategies to best verify the application's functionalities.

5.2 Implementation Plan

5.3 Component Integration and Testing

5.4 System Testing

5.5 Additional Specifications on Testing

6 EFFORT SPENT

Hours worked	Section 1	Section 2	Section 3	Section 4
Juan Sebastian Sarmiento Pulido	item 21	item 22	item 23	x
Pedro Augusto Villarinho dos Santos	item 21	item 22	item 23	x
Esteban Lopez Barreto	item 21	item 22	item 23	x

7 REFERENCES